講義 18：字串與日期處理（授課順序：接在講義 17 之後）

> 對應練習：ex18 | 答案程式：ZR_TR18_STRING_DATE

本講重點

- 字串五大指令：CONCATENATE、SPLIT、CONDENSE、REPLACE、TRANSLATE
- 位移取子字串 變數+位移(長度)、STRLEN( )
- c（固定長度）與 string（變動長度）處理上的差異
- 日期運算：加減天數、天數差、月初 / 月末、sy-datum / sy-uzeit

1. CONCATENATE：串接

```
DATA: gv_last  TYPE c LENGTH 10 VALUE '王',
      gv_first TYPE c LENGTH 10 VALUE '小明',
      gv_full  TYPE string,
      gv_path  TYPE string.

CONCATENATE gv_last gv_first INTO gv_full.
* gv_full = '王小明'——c 欄位「尾端」的空白串接時自動忽略

CONCATENATE 'A' 'B' 'C' INTO gv_path SEPARATED BY '-'.
* gv_path = 'A-B-C'——SEPARATED BY 指定分隔符
```

- 尾端空白被忽略是 c 型別的特性；**真的需要保留空白**時加 RESPECTING BLANKS（少用）。

新式寫法對照（7.40 之後）：字串模板（String Template）|...| 與 &&（串接運算子，Concatenation Operator）

```
* 傳統寫法
CONCATENATE gv_last gv_first INTO gv_full.
CONCATENATE 'A' 'B' 'C' INTO gv_path SEPARATED BY '-'.

* 新式寫法
CONCATENATE gv_last gv_first INTO DATA(gv_full2).   " 指令照舊，接收變數行內宣告（型別 string）
gv_full = |{ gv_last }{ gv_first }|.
gv_path = `A` && `-` && `B` && `-` && `C`.
```

語法拆解：

- **字串模板**：用 | 包起來的是一段字串，裡面的 { } 放變數或運算式，執行時換成它的值；{ } 以外的部分就是字面文字，例如 |全名：{ gv_last }{ gv_first }|。c 欄位的尾端空白在 { } 裡一樣被忽略，所以得到 王小明。

- `&&`：把左右兩段字串接起來，可以連續接。
- 好處：不用先準備接收變數，可以直接寫在 `WRITE`、`MESSAGE`、賦值右邊，文字與變數交錯時也比 `CONCATENATE` 好讀。
- `{ }` 裡還能加格式選項，例如 `{ gv_score WIDTH = 5 ALIGN = RIGHT PAD = '0' }` 補零成 `00085`、`{ gv_today DATE = ISO }` 印成 `2026-10-08`（完整清單見講義 26）。

2. SPLIT：拆解

```
DATA: gv_csv   TYPE string VALUE 'S0001,王小明,85',
      gv_id    TYPE string,
      gv_name  TYPE string,
      gv_score TYPE string.

SPLIT gv_csv AT ',' INTO gv_id gv_name gv_score.
```

- 接收變數比片段少：**最後一個變數收下剩餘全部**（含分隔符）。
- 片段數不固定時拆進內表（Internal Table）：`SPLIT gv_csv AT ',' INTO TABLE lt_parts.`——處理上傳檔案每一行的標準手法。

新式寫法對照（7.40 之後）：`segment( )`

```
* 傳統寫法：一次拆進多個變數
SPLIT gv_csv AT ',' INTO gv_id gv_name gv_score.

* 新式寫法：接收變數行內宣告（推導出的都是 string）
SPLIT gv_csv AT ',' INTO DATA(lv_id) DATA(lv_name) DATA(lv_score).

* 新式寫法：只取第 n 段
gv_name = segment( val = gv_csv index = 2 sep = ',' ).
```

語法拆解：`segment( val = 來源 index = 第幾段 sep = 分隔符 )` 是一個**字串函式（String Function）**，回傳第 `index` 段（從 1 起算）。函式寫法用 `參數名 = 值` 傳參數，可以直接放在賦值右邊或其他運算式裡。只要其中一段時用 `segment( )` 比較方便；要一次拆成好幾個變數，`SPLIT` 仍然最直接。

3. CONDENSE / REPLACE / TRANSLATE

```
DATA gv_text TYPE c LENGTH 30 VALUE '  ABAP   is   fun  '.

CONDENSE gv_text.                  " 'ABAP is fun'：去頭尾空白、連續空白縮成一個
CONDENSE gv_text NO-GAPS.          " 'ABAPisfun'   ：空白全部拿掉

REPLACE 'fun' WITH 'great' INTO gv_text.            " 只換第一個
REPLACE ALL OCCURRENCES OF 'a' IN gv_text WITH 'x'. " 全部換；沒找到 sy-subrc = 4

TRANSLATE gv_text TO UPPER CASE.  " 轉大寫（TO LOWER CASE 轉小寫）
```

實務備註：使用者輸入拿去跟資料庫比對前，常見前處理就是 `CONDENSE` + `TRANSLATE TO UPPER CASE`（資料庫多存大寫，講義 6 提過）。

新式寫法對照（7.40 之後）：字串函式（String Function）

傳統指令是「直接改掉變數本身」；字串函式則是「回傳處理後的結果」，原變數不動，要自己賦值回去：

```
* 傳統寫法
CONDENSE gv_text.
CONDENSE gv_text NO-GAPS.
REPLACE 'fun' WITH 'great' INTO gv_text.
TRANSLATE gv_text TO UPPER CASE.

* 新式寫法
gv_text = condense( gv_text ).
gv_text = condense( val = gv_text del = ` ` from = ` ` to = `` ).
gv_text = replace( val = gv_text sub = `fun` with = `great` ).
gv_text = to_upper( gv_text ).          " 轉小寫是 to_lower( )
```

語法拆解：

- 只有一個參數時可以直接寫值（`condense( gv_text )`、`to_upper( gv_text )`）；多個參數時一律寫 `參數名 = 值`。
- `condense( val = ... del = from = to = )` 等於 `NO-GAPS`：`del` 去掉頭尾的空白，`from` / `to` 把中間每個空白換成空字串。
- `replace( )` 預設只換第一個；要全部換加 `occ = 0`（等於 `ALL OCCURRENCES`）。
- **函式可以巢狀**：`gv_text = to_upper( replace( val = gv_text sub = fun with = great ) ).` 一行做完取代 + 轉大寫，傳統寫法要兩個指令。

練習 18 的新式寫法版本見答案程式 `ZR_TR18_STRING_DATE_NEW`（輸出跟原版逐行比對相同）。位移取值、`strlen( )` 與日期加減沒有新式寫法，兩版一樣。

4. 位移取子字串與 STRLEN

`變數+位移(長度)`：位移**從 0 起算**，這是 ABAP 少數從 0 數的地方：

```
DATA: gv_date_c TYPE c LENGTH 8 VALUE '20260705',
      gv_yyyy   TYPE c LENGTH 4,
      gv_id     TYPE c LENGTH 5 VALUE 'S0001',
      gv_len    TYPE i.

gv_yyyy = gv_date_c+0(4).         " '2026'：從第 0 位取 4 碼
WRITE: / gv_date_c+4(2).          " '07'   ：年月日各自切
gv_id+2(3) = '***'.               " 寫入也可以：'S0***'（遮罩效果）

gv_len = strlen( gv_date_c ).     " 8：字串長度（c 型別不含尾端空白）
```

- 超出實際長度的位移 / 長度組合會**執行期 dump**——對來源不定長的資料，先 `strlen( )` 檢查再切。
- 對 `string` 一樣可用（但 string 不能像 `c` 那樣「越界寫入」）。

5. c 與 string 的差異整理

	`c LENGTH n`	`string`
長度	固定，不足補空白	變動，多長存多長
尾端空白	一直都在（比較、CONCATENATE 時多半被忽略）	有就是有、沒有就是沒有
適用	固定格式欄位（代碼、學號、日期字串）	一般文字、拼接結果、檔案內容

最容易踩的差異：`c` 欄位塞超長內容**默默截斷**；`string` 不會。以及 `WRITE` 一個 `c LENGTH 20` 會佔滿 20 格版面，`string` 只佔實際長度。

6. 日期與時間

`d` 型別內容是 `YYYYMMDD` 的 8 碼字元，但**可以直接做整數式加減**——這是它的超能力：

```
DATA: gv_today TYPE d,
      gv_due   TYPE d,
      gv_first TYPE d,
      gv_last  TYPE d,
      gv_days  TYPE i.

gv_today = sy-datum.
gv_due   = gv_today + 30.            " 30 天後（自動跨月跨年）
gv_days  = gv_today - '20260101'.    " 兩個日期相減 = 相差天數

* 月初：把「日」的部分直接改成 01（位移寫入）
gv_first = gv_today.
gv_first+6(2) = '01'.

* 月末的標準套路：月初 + 31 天一定落在下個月 → 再改成下月 1 號 → 減 1 天
gv_last = gv_first + 31.
gv_last+6(2) = '01'.
gv_last = gv_last - 1.
```

- 拆年月日就用位移：`gv_today+0(4)` 年、`gv_today+4(2)` 月、`gv_today+6(2)` 日。
- **`WRITE` 日期要注意輸出長度（Output Length）**：自己宣告的 `TYPE d` 預設只佔 8 格，放不下分隔符號，`WRITE gv_today.` 印出 `20261008`；給 10 格（`WRITE (10) gv_today.`），或改用字典型別宣告（如 `TYPE sydatum`、`TYPE sflight-fldate`），就會依使用者設定加上分隔符號，例如 `2026/10/08`（2026-10-08 實測，`ZR_TR18_DATE_CHK`）。要固定格式就自己用位移拼，或用字串模板 `|{ gv_today DATE = ISO }|` 印成 `2026-10-08`。`WRITE ... TO` 的接收變數不能行內宣告（`WRITE gv_today TO DATA(gv_text)` 編譯錯誤），要事先宣告。
- 想要「跟畫面上 WRITE 出來一樣的格式」、但先放進變數再用：`WRITE gv_today TO gv_text.`（`gv_text` 是 `c LENGTH 10`）。`WRITE ... TO` 不輸出到清單，只把格式化後的文字放進變數，日期、

金額、數字都適用；加 `LEFT-JUSTIFIED` 可以靠左、去掉前面的空白（講義 13 的 ZRFI0004 用它組「頁碼 3/&」）。

- 時間 `t`（HHMMSS）同理可加減秒數；`sy-uzeit` 是現在時間。
- 星期幾、加「工作日」、民國年轉換等進階需求：標準 FM 都有（如 `DATE_COMPUTE_DAY`），講義 15 學會 CALL FUNCTION 後就能用——先記得「日期難題先找標準 FM」。

7. 常見錯誤與陷阱

症狀	原因
CONCATENATE 結果中間空白不見了	`c` 尾端空白自動忽略——要分隔用 SEPARATED BY
SPLIT 最後一個變數內容怪怪的	變數比片段少，剩餘全塞給最後一個
位移取值執行期 dump	位移+長度超過來源長度；且位移是 0 起算，別當成 1
REPLACE 後資料沒變	只換第一個（要 ALL OCCURRENCES OF），或大小寫不符
月末算成 30 號或 28 號	自己判斷大小月/閏年寫錯——用「下月初減一天」套路
日期相加結果變怪字串	其中一邊不是 `d` 型別（8 碼字元不等於日期）

8. 課堂練習

完成 ex18：姓名串接、CSV 拆欄、學號遮罩、字串整理（CONDENSE/REPLACE/TRANSLATE）、本月月初月末與天數差計算。